

JARVIS

ATIVACÃO

AUTOMÁTICA

Este documento é um **programa para Claude**. Cole no Claude Code na pasta do instalador, peça para ler e executar. Ao final, o JARVIS estará instalado, com os **14 patches anti-regressão** aplicados (10 originais + 4 post-launch), rodando em `localhost:3000` e respondendo comandos por voz e texto — sem nenhuma intervenção manual além de 1 login e 1 chave.

FASES	DURAÇÃO	PATCHES	OUTPUT
07	~17 min	14	JARVIS LIVE

AUTOR · GABRIEL FELIPE FERNANDES
RUNTIME · CLAUDE OPUS 4.7 · CLAUDE CODE
ATUALIZAÇÃO V8.1 · 25/04/2026 · 4
PATCHES POST-LAUNCH

J · A · R · V · I · S

INSTRUÇÕES PARA O CLAUDE

// LEIA ANTES DE
TUDO

Claude, este documento é um **plano de execução**. Sua missão: leia-o do início ao fim, depois execute as 7 fases na ordem. Ao final, o JARVIS precisa estar instalado, rodando, com os **14 patches** aplicados (10 da v8.0 + 4 da v8.1), e ter respondido aos 4 testes da Fase 06 com sucesso. Fale em **português brasileiro**, tom de amigo técnico, zero hesitação, zero "posso fazer?".

● REGRAS DE EXECUÇÃO

- ☐ **Execute em ordem sequencial.** Nunca pule uma fase.
- ☐ **Cada fase tem um GATE** — só avança quando o gate for verificado OK.
- ☐ **Se um passo falhar**, diagnostique pelo `install-log.txt`, conserte, retome. Não peça permissão.
- ☐ **Ações do tipo [EXECUTAR]** use as ferramentas Bash/Write/Read direto.
- ☐ **Ações do tipo [AGUARDAR]** explique ao usuário o que ele precisa fazer (ex.: login no Claude) e só avance quando ele responder "pronto".
- ☐ **Ações do tipo [DECLARAR]** anuncie o status em 1 frase antes de prosseguir.
- ☐ **Ao final**, entregue o sumário da Fase 07 literalmente como está.

● VISÃO GERAL DAS 7 FASES

01	02	03	04	05	06	07
PRÉ-CHECK	INSTALAÇÃO	VALIDAÇÃO	PATCH	ATIVAÇÃO	PROVA DE VIDA	ENTREGA

● FERRAMENTAS QUE VOCÊ VAI USAR

FASE	FERRAMENTA	PROPÓSITO
01	Bash · Read	Conferir presença de winget, admin, arquivos do instalador
02	Bash	Executar <code>INSTALAR-JARVIS-v8.bat</code> e monitorar <code>install-log.txt</code>
03	Bash · Read	Health check dos 7 componentes (Node, Git, Python, Claude, <code>server.js</code> , <code>node_modules</code> , <code>.env</code>)

FASE	FERRAMENTA	PROPÓSITO
04	Edit · Bash	Aplicar 14 patches anti-regressão (10 v8.0 + 4 v8.1) em server.js / public/script.js / pet/ ANTES de subir o servidor
05	Bash	Subir node server.js em janela visível, abrir browser em localhost:3000
06	Write · Bash	Criar arquivo de teste, validar que o sistema responde
07	Texto	Declarar conclusão



FASE 01

PRÉ-CHECK · Ambiente

GATE · 5/5 OK

EXECUTAR

Confira em paralelo:

- `where winget` retorna path
- `where node` (pode não existir ainda — OK)
- pasta atual tem `INSTALAR-JARVIS-v8.bat`
- usuário tem conexão com internet (`ping -n 1 github.com`)
- espaço em disco ≥ 10 GB

AGUARDAR

Pergunte ao usuário, numa mensagem só: "**Você tem a chave OpenAI (sk-...) copiada no clipboard? E está logado no Claude Pro?**" Só avance quando ele confirmar "sim".

DECLARAR

"Ambiente pronto. Vou disparar o instalador agora — ele leva uns 10 minutos. Enquanto ele roda, eu comento cada passo."

EXECUTAR Dispare o instalador como administrador em janela visível (Windows vai pedir UAC ao usuário):

▶ EXECUTAR

```
powershell -Command "Start-Process -FilePath 'INSTALAR-JARVIS-v8.bat' -Verb RunAs"
```

ACOMPANHAR A cada 30s, leia `install-log.txt` e anuncie o último passo concluído em 1 frase. Exemplo: "Passo 3 OK — Python instalado. Agora vai pro Claude CLI."

▶ EXECUTAR

```
tail -n 5 install-log.txt
```

AGUARDAR Quando o instalador pedir **login do Claude**, peça ao usuário para logar na janela que abrir. Quando pedir **chave OpenAI**, peça para colar a sk-... no terminal.

GATE Considere a fase concluída quando o log contém **FIM** e **RESULTADO: 7/7**. Se marcar menor, pule para o fallback abaixo.

FALLBACK Se der < 7/7: leia o log inteiro, identifique qual passo falhou (S1-S8), e execute o fix pontual. Git falhou → `winget install Git.Git`. Claude CLI falhou → `npm install -g @anthropic-ai/claude-code`. Depois retome a validação.

FASE 03

VALIDAÇÃO · 7 health checks

GATE · 7/7 green

EXECUTAR Rode os 7 checks em paralelo (uma mensagem, múltiplas chamadas Bash simultâneas):

#	CHECK	COMANDO
1	Git	git --version
2	Node	node --version (>= 18)
3	Python	"C:\Program Files\Python312\python.exe" --version
4	Claude CLI	claude --version
5	server.js	existe em %~dp0\server.js
6	node_modules	dir node_modules\express existe
7	.env	arquivo .env contém OPENAI_API_KEY=sk-

DECLARAR "Validação 7/7. Partes vivas. Antes de ligar, vou aplicar 14 patches que previnem bugs conhecidos da v8.0 e v8.1."

FALLBACK Se **.env** não tem chave: peça a chave ao usuário e escreva o arquivo. Se **node_modules** vazio: `npm install --production --no-audit` na pasta. Se **Claude CLI** não achar: `set PATH=%USERPROFILE%\local\bin;%PATH%` e retente.

EXECUTAR Aplicar os **14 patches** abaixo NA ORDEM, ANTES de subir `server.js`. Cada um previne um bug real detectado em produção. Patches **1-10** são da v8.0; **11-14** são post-launch v8.1 (descobertos 25/04/2026 no primeiro uso real). Use `Edit` e `Bash` direto — sem perguntar. As seções "CIRURGIA" abaixo das tabelas têm as instruções line-by-line de cada patch.

#	BUG	ARQUIVO · PATCH
1	PET não abre Zeus Electron ausente	<code>cd pet && npm install</code> (~300 MB, 404 pacotes)
2	Botão PET não liga mic SSE handler faltando	<code>public/script.js</code> · adicionar <code>case 'pet-mic'</code> no switch SSE → chamar <code>startRealtime()</code> quando <code>data.on=true</code>
3	Voz imprime código em vez de salvar Haiku ignora Write tool	<code>server.js</code> · <code>/api/voice-complete</code> e <code>/api/audio-complete</code> · forçar <code>acquireTaskProc({ model: 'sonnet' })</code>
4	"Pronto, senhor" não é falado notify nunca chamado	<code>server.js</code> · close handler de voice/audio · adicionar <code>await notifyBuildComplete(output, sse) + updateProjectStatus()</code>
5	Warm pool retorna vazio CLI sai com stdin vazio	<code>server.js</code> · <code>acquireTaskProc()</code> faz cold spawn (Claude nasce só com prompt); <code>_spawn()</code> usa <code>node cli.js</code> direto sem <code>cmd.exe</code>
6	Arquivos da raiz somem do HUD api só varre subpastas	<code>server.js</code> · <code>/api/files</code> · listar raiz + subpastas; arquivos soltos vão pro grupo (raiz)
7	GPT inventa "não foi possível" temperature alta + prompt otimista	<code>server.js</code> · <code>notifyBuildComplete()</code> · prompt fiel ao output, <code>temperature: 0.2</code> , fallback detecta <code>[file]/[error]/vazio</code>
8	Realtime fala em inglês mesmo com BR defaults 'EN' em 11 endpoints	<code>server.js</code> · trocar <code>language = 'EN' → 'BR'</code> em todos os endpoints; fallback <code>INSTRUCTIONS.BR</code> ; <code>whisper-1 'pt'</code> ; <code>INSTRUCTIONS.BR</code> com regra absoluta "Falar 'vou fazer' SEM chamar <code>execute_task = FALHA</code> "
9	"Claude não conseguiu — auth issue" warm pool morto + cli.js sem stdin	<code>server.js</code> · reverter <code>_spawn()</code> do warm pool pra <code>shell: true</code> com <code>CLAUDE_CMD</code> ; criar <code>spawnColdProc(model)</code> usando <code>node cli.js</code> direto; <code>/api/chat</code> usa <code>acquireTaskProc({model})</code> (cold spawn) ao invés de <code>acquireWithFallback</code>

#	BUG	ARQUIVO · PATCH
10	Mic captura mas GPT-realtime não chama execute_task áudio nunca chega ao /api/chat	public/script.js · no handler conversation.item.input_audio_transcription.completed, agendar fallback de 2s que dispara handleRealtimeTask(null, JSON.stringify({request: ev.transcript})) se o GPT não chamar a tool. Cancelar timer quando o GPT chamar execute_task. handleRealtimeTask só envia function_call_output se callId existir.

GATE Após cada patch: rode `grep` do trecho novo pra confirmar. Marque os 10 antes de avançar pra Fase 05.

FALLBACK Se um patch já estiver aplicado (commit anterior, instalação atualizada), pule pro próximo — NÃO duplique. Se conflito de linha, busque pelo nome da função (ex.: `notifyBuildComplete`) e edite o bloco existente.

CIRURGIA · 1 a 10 PATCH 1 (PET / Electron):

1. Bash: `cd "%IDIR%/pet" && npm install --no-audit --no-fund --loglevel=error` (~300 MB, 1-3 min).
2. Verifique: `ls pet/node_modules/electron/dist` deve existir.

PATCH 2 (botão PET liga mic):

1. Em `public/script.js` , dentro de `initNotifications()` → `es.onmessage` , troque o `if (payload.type === 'build-complete' ...)` por um `switch(payload.type)` com 2 cases: `'build-complete'` (mantém lógica original) e `'pet-mic'` (chama `startRealtime()` quando `payload.active === true` ; chama `stopRealtime()` quando false).

PATCH 3 (voz salva arquivo, não imprime código):

1. Em `server.js` , `/api/voice-complete` : substitua `proc = pools.haiku.acquire()` por `proc = acquireTaskProc({ model: 'sonnet' })` .
2. Em `/api/audio-complete` : substitua `const proc = pools.haiku.acquire()` por `const proc = acquireTaskProc({ model: 'sonnet' })` . Haiku ignora `Write` tool.

PATCH 4 ("Pronto, senhor" no fim):

1. Em `server.js` , no `proc.on('close', () => { ... })` de `/api/voice-complete` (após `res.end()`), adicione: `try { notifyBuildComplete(message, responseBuffer, voiceLang); } catch(e) {} updateProjectStatus(message, responseBuffer).catch(()=>{})`
2. Mesma adição no `proc.on('close')` de `/api/audio-complete` (use `req.body.language || 'BR'`).

PATCH 5 (cold spawn dedicado — infra pra 3 e 9):

1. Em `server.js` , logo após `let CLAUDE_CMD = 'claude'` , adicione `findClaudeCliJs()` que checa estes paths e retorna o primeiro existente:
`%USERPROFILE%/AppData/Roaming/npm/node_modules/@anthropic-ai/claude-code/cli.js` ,
`%USERPROFILE%/.local/lib/node_modules/@anthropic-ai/claude-code/cli.js` , `C:\Program Files\nodejs\node_modules\@anthropic-ai\claude-code\cli.js` .
Atribua a `const CLAUDE_CLI_JS = findClaudeCliJs()` .
2. Adicione `spawnColdProc(model)` : deduz `modelId` a partir do slug ('opus'/'sonnet'/'haiku'), monta `args = ['--print', '--output-`

- ```
format', 'text', '--model', modelId, '--dangerously-skip-
permissions'] . Se CLAUDE_CLI_JS existir:
spawn(process.execPath, [CLAUDE_CLI_JS, ...args], { shell:
false, cwd: JARVIS_DIR, windowsHide: true }) . Senão fallback:
spawn(CLAUDE_CMD, args, { shell: true, cwd: JARVIS_DIR }) .
```
3. Adicione `acquireTaskProc({ model = 'sonnet' } = {})` : retorna `null` se `!claudeCliAvailable` , senão `spawnColdProc(model)` .
  4. **NÃO** mexa em `_spawn()` da classe `WarmPool` : ele continua com `spawn(CLAUDE_CMD, [...flags], { shell: true, cwd: JARVIS_DIR })` . Sem TTY o cli.js morre antes do `acquire`.

#### PATCH 6 (arquivos da raiz no HUD):

1. Em `server.js` , `/api/files` : além de iterar subpastas, leia `fs.readdirSync(PROJECTS_DIR, {withFileTypes:true})` e, para cada `entry.isFile()` com extensão em `deliverableExts` , push do objeto `{ name, project: '(raiz)', path, size, ext, createdAt, downloadUrl }` . Ignore arquivos com `name.startsWith('.')` .

#### PATCH 7 (notify fiel ao output):

1. Em `server.js` , `notifyBuildComplete()` : troque `temperature: 0.8` por `temperature: 0.2` .
2. No início da função, adicione fallback determinístico: `const trimmed = (claudeResponse||'').trim(); if (!trimmed || trimmed.length < 8 || /\[error\]/i.test(trimmed)) { pushNotification({type:'build-complete', message: fallback, language}); return; }`
3. Após o `Promise.race` , antes de pushar: detecte fabricação — `const looksFabricated = rich && /n[âa]o\s+foi\s+poss[ii]vel|unable\s+to|couldn't/i.test(rich) && /\[error\]/i.test(trimmed); const final = (looksFabricated ? null : rich) || fallback;`

#### PATCH 8 (default BR + Realtime forte):

1. Em `server.js` , `replace_all: language = 'EN'` → `language = 'BR'` (~10 endpoints/funções).
2. Troque `|| INSTRUCTIONS.EN` por `|| INSTRUCTIONS.BR` (1 ocorrência, em `/api/realtime/session` ).
3. Troque `[language] || 'en'` por `[language] || 'pt'` (1 ocorrência, whisper-1 fallback).
4. Em `INSTRUCTIONS.BR` , reforçe: substitua "REGRA ABSOLUTA: ... → chame "execute\_task". Sem exceção." por "REGRA ABSOLUTA: ... → chame IMEDIATAMENTE a função "execute\_task" no MESMO turno. Falar "vou fazer", "já estou fazendo", "um momento" SEM chamar `execute_task` no mesmo turno = FALHA."

#### PATCH 9 (/api/chat usa cold spawn):

1. Em `server.js` , `/api/chat` , depois de `const model = selectModelByComplexity(...)` : troque `const proc = acquireWithFallback(model);` por `const proc = acquireTaskProc({ model }) || acquireWithFallback(model);`  
Cold spawn é primário (warm pool morre silenciosamente entre `_spawn` e `acquire`), warm pool fica como último recurso.

#### PATCH 10 (fallback de execução do mic + deduplicação):

1. Em `public/script.js` , dentro de `realtimeDC.addEventListener('message', ...)` : localize o handler de `ev.type === 'conversation.item.input_audio_transcription.completed'` e adicione no fim, depois do `addTerminalLine` :

```
window._pendingTranscript = ev.transcript;
window._dispatchedThisTurn = false; // novo turno
clearTimeout(window._pendingTranscriptTimer);
window._pendingTranscriptTimer = setTimeout(() => {
 if (window._dispatchedThisTurn) return; // GPT-realtime cuidou
 const t = window._pendingTranscript;
```

```

 if (!t) return;
 window._pendingTranscript = null;
 window._dispatchedThisTurn = true;
 handleRealtimeTask(null, JSON.stringify({ request: t }));
 }, 2000);

```

2. No handler de `ev.type` `===`

`'response.function_call_arguments.done' && ev.name` `===`  
`'execute_task'` , substitua o conteúdo por:

```

window._pendingTranscript = null;
clearTimeout(window._pendingTranscriptTimer);
if (window._dispatchedThisTurn) return; // dedup: fallback já
disparou
window._dispatchedThisTurn = true;
handleRealtimeTask(ev.call_id, ev.arguments);

```

3. Em `handleRealtimeTask(callId, argsJson)` , troque `if`  
`(realtimeDC?.readyState === 'open') {` por `if (callId &&`  
`realtimeDC?.readyState === 'open') {` — o `function_call_output`  
exige `call_id` real; no fallback `callId` vem `null` e seria rejeitado.

4. **Por quê dedup?** Sem ela, fallback dispara em 2s E o GPT-realtime pode  
chamar a tool depois — gerando 2 chamadas `/api/chat` . A segunda  
chamada pega Claude ocupado/sem `stdin`, sai vazia → push notification  
"auth issue" mesmo após a primeira ter completado a tarefa.

Estes 10 patches são auto-contidos no roteiro. Aplique via `Edit` / `Bash` direto.  
Não há script externo necessário.

**DECLARAR** "10 patches v8.0 aplicados. Avançando para os 4 patches post-launch v8.1."



**EXECUTAR** 4 bugs descobertos em produção em **25/04/2026** durante o primeiro uso real do JARVIS instalado. Aplicar APÓS os 10 patches da v8.0.

| #  | BUG                                                                                                                    | ARQUIVO - PATCH                                                                                                                                                                      |
|----|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 11 | Build morto quando frontend fecha conexão<br>req.on('close') matava o proc;<br>push fake "Claude não conseguiu"        | server.js - /api/chat - flag clientAlive + killTimer 300s + code !== null silencioso; build segue em background mesmo se cliente fechou                                              |
| 12 | Haiku roteado para execução de tarefas<br>Haiku ignora Write tool; tarefa termina sem criar arquivo                    | server.js - selectModelByComplexity default → Sonnet; AGENT_MODEL_MAP['sm'] → Sonnet; acquireWithFallback sem Haiku; cold spawn de Sonnet como último recurso                        |
| 13 | Voz Realtime parafraseia anúncio de fim<br>GPT-realtime adiciona personalidade própria mesmo instruído a falar literal | public/script.js - build-complete sempre via speakResponse() (TTS) — fala palavra por palavra o resumo do GPT-mini, sem passar pelo Realtime                                         |
| 14 | Mensagens user-facing citam "Claude"<br>quebra ilusão de assistente autônomo                                           | server.js + public/script.js - todas as strings de erro/notificação refeitas em 1ª pessoa: "sistema de execução", "instabilidade", "Erro de execução" — JARVIS assume todas as ações |

CIRURGIA - 11 a 14

PATCH 11 (Build sobrevive a client disconnect):

1. Em /api/chat , killTimer : 120000 → 300000 (5 min para HTML completo).

2. Substituir req.on('close', () ⇒ { try { proc.kill(); } catch {} ... }) por flag let clientAlive = true; req.on('close', () ⇒ { clientAlive = false; }) .

3. Em proc.stdout.on('data') : if (clientAlive) { try { res.write(chunk); } catch {} } .

4. No proc.on('close', (code) ⇒ ...) :trocar if (!responseBuffer.trim() && code !== 0) por if (!responseBuffer.trim() && code !== 0 && code !== null) — exit null = killed by us, não disparar push fake.

5. Envolver os res.end() e res.write() finais com if (clientAlive) .

PATCH 12 (Haiku banido como executor):

1. Em selectModelByComplexity : adicionar cria na regex Sonnet (era só crie ). Default final: return 'claude-sonnet-4-6' (era claude-haiku-4-5-20251001 ).

2. Em AGENT\_MODEL\_MAP : 'sm': 'claude-haiku-4-5-20251001' → 'sm': 'claude-sonnet-4-6' .

3. Em acquireWithFallback : tier0Order sem pools.haiku .Opus → [opus, sonnet] ; Sonnet → [sonnet] . Last resort: spawnColdProc('sonnet') (era pools.haiku.\_spawn() ).

4. Warm pool de Haiku continua existindo (usada em /api/voice-spawn pre-spawn), mas nenhum endpoint de execução real roteia para ela.

PATCH 13 (Voz fim-de-task usa TTS, não Realtime):

1. Em `public/script.js`, no `case 'build-complete'` de `initNotifications()` : remover o branch `if (realtimeActive) announceToRealtime(...)` .
2. Manter apenas: `if (userGestureReceived) speakResponse(payload.message);` — TTS lê literal, sem parafraseio do GPT-realtime.
3. **Por quê?** Mesmo instruído a "falar exato esta frase", o GPT-realtime adiciona personalidade própria — voz fica desconectada do que foi feito. TTS fala palavra por palavra.

#### **PATCH 14 (JARVIS fala em 1ª pessoa, nunca cita Claude/GPT/OpenAI):**

1. **Prompt do GPT-mini** em `notifyBuildComplete` (BR): trocar *"FIEL ao output do Claude — descreva apenas o que ELE realmente fez"* por *"FIEL ao output gerado — descreva apenas o que foi realmente feito"*. Adicionar no fim: *"NUNCA cite 'Claude', 'GPT', 'OpenAI' ou qualquer ferramenta interna — fale como se VOCÊ tivesse feito."* Trocar `Claude's output (summary):` por `Output (summary):` .
2. **Mensagens de erro** `/api/chat` : `"Claude Code não está disponível"` → `"Sistema de execução temporariamente indisponível"` ; `"Senhor, o Claude Code não está configurado"` → `"Senhor, meu sistema de execução não está configurado"` ; `"Claude process pool exhausted"` → `"Sistema sobrecarregado no momento"` ; `"Senhor, o Claude não conseguiu executar"` → `"Senhor, encontrei uma instabilidade processando essa tarefa"` ; `"[error] Claude exited with code"` → `"[error] Execução interrompida (code=...)"` ; `"[error] Claude CLI error"` → `"[error] Erro de execução"` .
3. `/api/voice-complete` + `/api/audio-complete` : `"Claude Code is not configured..."` → `"Sistema de execução não configurado..."` ; `"[error] Claude process unavailable"` → `"[error] Sistema de execução indisponível"` (replace\_all); `"[error] Claude Code not configured"` → `"[error] Sistema de execução não configurado"` .
4. `public/script.js` : `"Claude is executing in background"` → `"Task is executing in background"` ; `"Acionando Claude para corrigir"` → `"Acionando sistema para corrigir"` .
5. **Logs internos** ( `console.log` / `console.error` ) PODEM continuar citando Claude — são logs de servidor, não chegam ao usuário.

Estes 4 patches v8.1 são auto-contidos. Aplique via `Edit` / `Bash` direto. Origem: sessão de debug 25/04/2026 — Gabriel Felipe Fernandes + Claude Opus 4.7.

#### **GATE**

Após os 4 patches v8.1: `node --check server.js` . Confirme via grep: `"build continues in background"` , `"spawnColdProc('sonnet')"` , `"ALWAYS use TTS for build-complete"` , `"NUNCA cite \"Claude\""` . Marque os 14 antes de avançar pra Fase 05.

#### **DECLARAR**

"14 patches aplicados (10 v8.0 + 4 v8.1). Build sobrevive a disconnect, Haiku banido da execução, voz fala literal, JARVIS assume todas as ações em 1ª pessoa. Subindo agora."

## FASE 05

## ATIVAÇÃO · Subir o servidor

GATE · HTTP 200 em :3000

**EXECUTAR** Mate qualquer processo na porta 3000, suba o server em janela visível, abra o browser:

## ▶ EXECUTAR

```
for /f "tokens=5" %p in ('netstat -ano ^| findstr ":3000.*LISTENING"') do taskkill /F /PID %p
start "JARVIS" cmd /k "node server.js"
timeout /t 5 /nobreak >nul
start "" "http://localhost:3000"
```

**GATE** Após 5s: `curl -s http://localhost:3000/api/health` deve retornar `{"status": "ok"}` ou similar, com `pools: { opus, sonnet, haiku } >= 1` cada.

**DECLARAR** "JARVIS online em *localhost:3000*. HUD carregando no navegador. Vamos testar."

## FASE 06

## PROVA DE VIDA · primeiro teste

GATE · 4/4 tarefas

**EXECUTAR** Execute você mesmo os 4 testes do sistema — NÃO peça para o usuário colar no chat do JARVIS. Você É o JARVIS agora.

## TESTE 1 · HORA ATUAL

**AÇÃO** Responda com a hora local do Windows no fuso do usuário (use `date /t + time /t`).

## TESTE 2 · CRIAR ARQUIVO DE EVIDÊNCIA

**AÇÃO** Crie o arquivo `Documents and Projects\JARVIS-ATIVO.txt` com conteúdo padronizado:

## ▶ CRIAR ARQUIVO

```
JARVIS v8.1 — ATIVADO COM SUCESSO
Data: {data_hora_atual}
Componentes: Git · Node · Python · Claude CLI · Obsidian · Express · OpenAI
Patches: 14/14 (10 v8.0 + 4 v8.1)
Porta: 3000
Status: ONLINE
```

Se você está lendo este arquivo, seu JARVIS está pronto para qualquer tarefa.

## TESTE 3 · CLIMA

**AÇÃO** Chame `GET http://localhost:3000/api/weather` e declare o clima em 1 frase.

## TESTE 4 · CAPACIDADES (RESPOSTA TEXTUAL)

**DECLARAR** "Posso controlar seu PC, criar qualquer tipo de arquivo ou app, responder por voz e manter memória permanente via Obsidian — tudo local."

**DECLARAR** Envie exatamente este formato ao usuário:

▶ EXECUTAR

✓ JARVIS v8.1 • ATIVO

- Servidor · http://localhost:3000
- HUD · aberto no navegador
- Voz · OpenAI conectado (TTS literal · v8.1)
- Memória · Obsidian vault em Documentos\Felipe
- Atalho · Ligar JARVIS no Desktop
- Patches · 14/14 aplicados (anti-regressão)
- Evidência · Documents and Projects\JARVIS-ATIVO.txt

Próximo passo: me pede qualquer coisa.

Ex: "cria uma landing page pro meu curso"

Ex: "abre o YouTube e toca lofi"

Ex: "faz uma planilha de controle financeiro"

## COMO DISPARAR ISSO // PROMPT ÚNICO

Abra o **Claude Code** (Claude Desktop → Ctrl+3), navegue até a pasta **Instalador Jarvis** (esta pasta onde este PDF está). Cole o texto abaixo no chat e aperte Enter. A partir daí o Claude assume tudo até o JARVIS estar ativo.

### ► PROMPT INICIAL

Claude, leia o arquivo ROTEIRO-AULA-JARVIS-v8.1.pdf nesta pasta. Ele é um documento executável – siga as 7 fases dele na ordem até o JARVIS estar instalado, com os 14 patches da Fase 04 aplicados (10 v8.0 + 4 v8.1), rodando em localhost:3000, e ter passado nos 4 testes da Fase 06. Fale em português BR, tom de amigo técnico, zero hesitação. Quando terminar, entregue o sumário da Fase 07 literal. Comece agora.

## ● PROMPT DE CONTINGÊNCIA

Se algo travar no meio e você quiser que o Claude diagnostique e retome:

### ► PROMPT INICIAL

Pausa. Algo travou. Lê install-log.txt, identifica qual fase falhou, conserta, e retoma de onde parou – entregando o sumário da Fase 07 no final. Sem perguntar, só faz.

## ● PROMPT PARA REINSTALAR DO ZERO

Se quiser apagar tudo e começar de novo:

### ► PROMPT INICIAL

Mata qualquer JARVIS rodando, apaga node\_modules, apaga .env, e roda de novo o ROTEIRO-AULA-JARVIS-v8.1.pdf inteiro do zero (incluindo os 14 patches da Fase 04). Me entrega o sumário final quando terminar.

## ● ARQUIVOS QUE ESTE DOCUMENTO CRIA OU USA

LÊ

### INSTALAR-v8.bat

Instalador automatizado (não modifique).

LÊ

### install-log.txt

Gerado pelo .bat — usado pelo Claude para monitorar.

CRIA

### JARVIS-ATIVO.txt

Arquivo de evidência no final da Fase 05.

# READ · EXECUTE · DELIVERED

Este documento é um programa. Você, Claude, é o interpretador. O ponto final é o JARVIS rodando, 14 patches aplicados, e o sumário da Fase 07 na tela do usuário.

J·A·R·V·I·S · V8.1 · 2026 · DOCUMENTO EXECUTÁVEL · GABRIEL FELIPE FERNANDES

